

Item 01

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.1. Documentação de APIs
 - **Referência:** Aula 01
-

Capacidade: CT01 - Reconhecer as especificações dos serviços disponíveis no servidor

Contexto: Durante a análise da documentação técnica de uma API de logística, um desenvolvedor observa o endereço `/api/cep/v1/{cep}` e precisa identificar a forma como a informação do CEP é transmitida no caminho do serviço.

Comando: Identifique a denominação dos parâmetros obrigatórios inseridos diretamente na estrutura do caminho da URL e delimitados por chaves na documentação técnica.

Alternativas:

- ☐ A) Query Parameters
 - ☐ B) Path Parameters
 - ☐ C) Header Parameters
 - ☐ D) Payload Body
 - ☐ E) Status Codes
-

Gabarito: B

Justificativa: Os *Path Parameters* são identificadores obrigatórios inseridos diretamente na estrutura de caminhos da URL (ex: `/api/cep/v1/{cep}`), diferenciando-se dos parâmetros de consulta (*Query Parameters*) anexados ao final da URL após o sinal `?`.

Distratores:

- A) Os *Query Parameters* são filtros ou modificadores opcionais anexados ao final da URL após o sinal `?` (ex: `/encomendas?status=Pendente`).
 - B) Alternativa correta.
 - C) Os *Header Parameters* representam metadados de contexto enviados no cabeçalho da requisição (ex: `Content-Type`).
 - D) O *Payload Body* refere-se ao corpo estruturado da mensagem enviado em requisições como POST ou PUT.
 - E) Os *Status Codes* constituem as respostas numéricas de três dígitos enviadas pelo servidor para relatar o resultado do processamento.
-

Item 02

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.3. Tratamento de mensagens do server-side
 - **Referência:** Aula 02
-

Capacidade: CT05 - Aplicar tratamento de falhas nas mensagens do servidor

Contexto: Um analista de suporte monitora a integração entre a interface do cliente e o banco de dados. Ao disparar uma consulta, o servidor sofre uma queda inesperada no banco de dados e retorna uma resposta com código numérico pertencente à família 5xx.

Comando: Identifique o significado técnico das mensagens de status HTTP pertencentes à família de códigos 5xx.

Alternativas:

- ☐ A) Informações sobre requisição recebida e em processamento.
 - ☐ B) Confirmação de que a ação foi aceita e processada com sucesso.
 - ☐ C) Indicação de que ações adicionais ou uso de cache são necessários.
 - ☐ D) Sinalização de erro de sintaxe ou dados inválidos enviados pelo cliente.
 - ☐ E) Indicação de falha interna ocorrida no servidor ao processar o pedido.
-

Gabarito: E

Justificativa: A família de códigos 5xx (Erro do Servidor) sinaliza que a requisição enviada pelo cliente era válida, mas o servidor falhou internamente ao tentar processá-la (ex: erro 500 Internal Server Error ou 503 Service Unavailable).

Distratores:

- A) A família 1xx (Informativas) indica que a requisição foi recebida e continua em processamento.
 - B) A família 2xx (Sucesso) confirma que a requisição foi recebida, compreendida e aceita com êxito.
 - C) A família 3xx (Redirecionamento/Cache) indica a necessidade de ações adicionais para concluir a requisição ou o reaproveitamento de cache.
 - D) A família 4xx (Erro do Cliente) sinaliza requisições sintaticamente incorretas ou recursos não encontrados.
 - E) Alternativa correta.
-

Item 03

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.3. Tratamento de mensagens do server-side
 - **Referência:** Aula 02
-

Capacidade: CT05 - Aplicar tratamento de falhas nas mensagens do servidor

Contexto: Durante a navegação em um painel web, o navegador envia uma requisição condicional ao servidor para verificar se uma tabela de dados sofreu alterações e recebe como resposta o código de status HTTP 304 Not Modified.

Comando: Determine a ação realizada pelo navegador web ao receber o código de status HTTP 304 Not Modified.

Alternativas:

- ☐ A) Exibir uma mensagem de erro de página não encontrada ao usuário.
 - ☐ B) Reaproveitar a cópia do recurso armazenada no cache local.
 - ☐ C) Apagar permanentemente os dados salvos na memória do localStorage.
 - ☐ D) Cancelar a execução do script e congelar a interface visual.
 - ☐ E) Forçar a criação de um novo registro duplicado no banco de dados.
-

Gabarito: B

Justificativa: O código 304 Not Modified indica que o recurso solicitado não sofreu alterações no servidor desde o último acesso, instruindo o navegador a reaproveitar a cópia salva no *cache* local para economizar largura de banda.

Distratores:

- A) A mensagem de página não encontrada está associada ao código de erro 404 Not Found.
- B) Alternativa correta.
- C) O código 304 orienta o uso do cache HTTP do navegador, não interferindo na limpeza manual do *localStorage*.
- D) O código 304 é uma resposta válida de otimização de rede, não um erro de travamento de script.

- E) A criação de registros é realizada via requisição POST (retornando status 201 Created), e não por respostas de cache.
-

Item 04

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 03
-

Capacidade: CT02 - Reconhecer os serviços disponíveis no servidor

Contexto: Ao integrar a consulta de informações em um painel de monitoramento, o programador executa a instrução `fetch("https://brasilapi.com.br/api/ddd/v1/47")` sem especificar parâmetros adicionais de configuração no segundo argumento.

Comando: Indique o método HTTP executado por padrão pela função `fetch()` quando nenhum verbo é explicitamente configurado.

Alternativas:

- ☐ A) POST
 - ☐ B) PUT
 - ☐ C) GET
 - ☐ D) DELETE
 - ☐ E) PATCH
-

Gabarito: C

Justificativa: A Fetch API é a ferramenta padrão do navegador para requisições HTTP e, por padrão, ao chamar `fetch(url)` sem objetos de configuração extras, o navegador executa uma requisição utilizando o método **GET** (operação de Consulta).

Distratores:

- A) O método POST requer a configuração explícita da propriedade `method: "POST"` no objeto de opções da Fetch API.
 - B) O método PUT exige declaração explícita no objeto de opções do `fetch`.
 - C) Alternativa correta.
 - D) O método DELETE exige configuração explícita de `method: "DELETE"` no envio da requisição.
 - E) O método PATCH precisa ser configurado manualmente no objeto de parâmetros do `fetch`.
-

Item 05

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.3. Tratamento de mensagens do server-side
 - **Referência:** Aula 03
-

Capacidade: CT05 - Aplicar tratamento de falhas nas mensagens do servidor

Contexto: Para garantir que o painel web remova o indicador visual de carregamento ("Carregando...") ao término da consulta à API, o desenvolvedor envolveu o código da requisição HTTP em uma estrutura de controle `try/catch/finally`.

Comando: Identifique o momento de execução das instruções contidas dentro do bloco `finally` na estrutura de controle `try/catch/finally`.

Alternativas:

- ☐ A) Apenas quando a requisição de rede retorna um código de sucesso 200 OK.
 - ☐ B) Apenas quando ocorre uma exceção ou falha capturada pelo bloco catch.
 - ☐ C) Antes do início da execução das instruções presentes no bloco try.
 - ☐ D) Obrigatoriamente ao término da operação, ocorrendo sucesso ou falha.
 - ☐ E) Exclusivamente se o servidor responder com status da família 4xx.
-

Gabarito: D

Justificativa: O bloco **finally** contém o código de limpeza e finalização que é executado obrigatoriamente ao término da operação assíncrona, independentemente de ela ter sido concluída com sucesso (bloco **try**) ou com erro (bloco **catch**).

Distratores:

- A) O código de sucesso é processado no bloco **try**, enquanto o **finally** roda ao final em todos os cenários.
 - B) A captura e o tratamento de falhas cabem exclusivamente ao bloco **catch**.
 - C) O bloco **try** é o primeiro a ser executado no fluxo de controle.
 - D) Alternativa correta.
 - E) Respostas de erro da família 4xx acionam fluxos de tratamento de falha, mas o **finally** roda indistintamente ao fim de qualquer execução.
-

Item 06

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 04
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: O painel de controle do sistema de entregas precisa permitir que os operadores alterem o status de um pacote para "ENTREGUE" e removam encomendas canceladas do sistema.

Comando: Identifique os métodos HTTP correspondentes, respectivamente, às operações do ciclo CRUD de Atualização e Exclusão de registros.

Alternativas:

- ☐ A) GET e POST
 - ☐ B) POST e GET
 - ☐ C) PUT/PATCH e DELETE
 - ☐ D) DELETE e PUT
 - ☐ E) POST e DELETE
-

Gabarito: C

Justificativa: No ciclo CRUD, as operações de Atualização de registros são mapeadas pelos verbos HTTP **PUT** ou **PATCH**, enquanto a operação de Exclusão/Destruição é realizada pelo verbo **DELETE**.

Distratores:

- A) O método GET realiza a Consulta (Read) e o POST executa a Criação (Create).
 - B) O método POST executa a Criação (Create) e o GET realiza a Consulta (Read).
 - C) Alternativa correta.
 - D) A ordem apresentada inverte as operações, colocando Exclusão antes de Atualização.
 - E) O método POST realiza a Criação de novos registros, e não a Atualização de registros já existentes.
-

Item 07

- **Dificuldade:** Fácil

- **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 04
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Ao refatorar uma aplicação de monitoramento de encomendas para a arquitetura de *Single Page Application* (SPA), a equipe de desenvolvimento eliminou a necessidade de recarregar a tela a cada alteração efetuada nos dados.

Comando: Indique o principal benefício de Experiência do Usuário (UX) proporcionado pela arquitetura de *Single Page Application* (SPA) no envio e manipulação de formulários.

Alternativas:

- ☐ A) Atualização dinâmica de conteúdos na tela sem o recarregamento completo da página.
 - ☐ B) Execução síncrona de scripts com congelamento da pilha de execução.
 - ☐ C) Eliminação da necessidade de utilizar conexões via protocolo HTTP/HTTPS.
 - ☐ D) Permissão para alteração direta de arquivos físicos salvos no computador local.
 - ☐ E) Bloqueio permanente de componentes visuais e seletores interativos na interface.
-

Gabarito: A

Justificativa: A arquitetura SPA (Single Page Application) permite que a interface envie e receba dados em segundo plano e atualize partes específicas da tela dinamicamente sem a necessidade de a página piscar ou recarregar inteiramente (sem "F5" forçado).

Distratores:

- A) Alternativa correta.
 - B) O assincronismo das SPAs busca exatamente evitar a execução bloqueante e o congelamento da interface.
 - C) Aplicações SPA continuam dependendo do protocolo HTTP/HTTPS em ambiente de rede para trocar dados com APIs.
 - D) O navegador impede o acesso e alteração direta a arquivos locais no disco do computador por regras de segurança da web.
 - E) A arquitetura SPA busca aumentar a interatividade da interface, e não bloqueá-la.
-

Item 08

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 05
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Um desenvolvedor web está formatando uma mensagem para exibir na tela o nome do cliente e o código de rastreio da encomenda utilizando a sintaxe de crases (``) e o operador `\${}`.

Comando: Identifique o recurso do JavaScript moderno utilizado para realizar a interpolação de variáveis e quebras de linha em textos de forma concisa.

Alternativas:

- ☐ A) Template Strings
- ☐ B) Callbacks
- ☐ C) Event Loop
- ☐ D) Arrow Functions
- ☐ E) Promises

Gabarito: A

Justificativa: As *Template Strings* (delimitadas por crases ``) representam a forma moderna de manipular textos em JavaScript, permitindo quebras de linha e interpolação direta de variáveis via `${variavel}`.

Distratores:

- A) Alternativa correta.
 - B) *Callbacks* são funções passadas como argumentos para outras funções para execução posterior.
 - C) O *Event Loop* é o mecanismo do navegador encarregado de entregar os resultados de tarefas assíncronas de volta à pilha de execução.
 - D) *Arrow Functions* representam uma sintaxe curta para declaração de funções anônimas (`=>`).
 - E) *Promises* são objetos que representam a conclusão ou falha de uma operação assíncrona futura.
-

Item 09

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 05
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Ao comparar o código de um filtro digitado com o valor numérico retornado pela API, o programador evitou o uso do operador de igualdade ampla (`==`) e optou pelo operador de igualdade estrita (`===`).

Comando: Indique a característica do operador de igualdade estrita (`===`) em relação ao operador de igualdade ampla (`==`).

Alternativas:

- ☐ A) Realizar a conversão automática de tipo de dado antes de avaliar a comparação.
 - ☐ B) Comparar o valor e o tipo de dado sem realizar coerção automática de tipo.
 - ☐ C) Retornar sempre um valor do tipo Number em vez de um resultado Booleano.
 - ☐ D) Permitir a comparação exclusiva entre variáveis declaradas com a palavra `var`.
 - ☐ E) Inverter o valor booleano do resultado obtido na expressão lógica.
-

Gabarito: B

Justificativa: O operador de igualdade estrita (`===`) compara o valor e o tipo de dado das variáveis envolvidas sem realizar a conversão (coerção) de tipo automática (ex: `'5' === 5` retorna `false`, prevenindo *bugs*).

Distratores:

- A) A conversão (coerção) automática de tipo ocorre no operador de igualdade ampla (`==`), o que pode gerar resultados inesperados como `'5' == 5` retornar `true`.
 - B) Alternativa correta.
 - C) Os operadores de comparação sempre retornam um valor booleano (`true` ou `false`).
 - D) O operador de igualdade estrita funciona com qualquer variável, independente de ser declarada com `const`, `let` ou `var`.
 - E) A inversão de valor booleano é realizada pelo operador lógico de negação (`!`).
-

Item 10

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.1. Documentação de APIs
 - **Referência:** Aula 05
-

Capacidade: CT01 - Reconhecer as especificações dos serviços disponíveis no servidor

Contexto: Uma equipe de desenvolvimento criou uma biblioteca com funções de validação no arquivo `utils.js` e precisa disponibilizar essas funções para consumo no arquivo principal `main.js`.

Comando: Identifique as declarações do JavaScript moderno (ES6+) utilizadas, respectivamente, para disponibilizar uma função em um módulo e para consumi-la em outro arquivo.

Alternativas:

- ☐ A) `push` e `pop`
 - ☐ B) `export` e `import`
 - ☐ C) `save` e `load`
 - ☐ D) `send` e `receive`
 - ☐ E) `set` e `get`
-

Gabarito: B

Justificativa: No sistema nativo de módulos do JavaScript moderno (ES6+), a instrução `export` disponibiliza variáveis ou funções de um arquivo módulo, enquanto a instrução `import` as carrega em outro arquivo do projeto.

Distratores:

- A) `push` e `pop` são métodos de inclusão e remoção de elementos no final de um Array.
 - B) Alternativa correta.
 - C) `save` e `load` não são palavras-chave de manipulação de módulos na sintaxe padrão do ES6+.
 - D) `send` e `receive` são termos comuns em comunicação de sockets/rede, não declarações de módulos JS.
 - E) `set` e `get` são palavras-chave usadas na definição de propriedades de acesso em objetos/classes ou estruturas do tipo Set.
-